

Kinematic Restitution Balancing (KRB)

A Per-Constraint Energy Audit for Velocity-Level Impulse Solvers

Joshua Sideris

Gear3Games

<https://gearbox2d.com>

josh.sideris@gmail.com

Abstract

In discrete physics simulations using velocity-level impulse solvers (Sequential Impulse / Projected Gauss-Seidel), energy gain is a common numerical artifact. This paper introduces **Kinematic Restitution Balancing (KRB)**, a per-constraint correction for that leak in both unilateral constraints (collisions) and bilateral constraints (joints). By compensating for force-induced velocity drift (Component A) and auditing potential-energy shifts during position correction (Component B), KRB audits and cancels the two usual analytic SI energy leaks inside a single-pass SI/PGS pipeline, without a second position pass or extra PGS iterations. The per-constraint audit does not claim a global energy invariant; coupled constraints (e.g. Newton’s cradle offsets) can still exchange systemic potential-energy work.

1. Introduction

Velocity-level sequential-impulse (SI) solvers [Catto 2005] are the standard real-time rigid-body approach: constraints are solved after explicit force integration with a fixed iteration count. KRB is a drop-in *preSolve* energy audit for that pipeline. It does not add solver passes, replace PGS, or switch to position-based dynamics.

Bender, Erleben, and Trinkle survey interactive rigid-body practice [Bender et al. 2014]. Baumgarte stabilization [Baumgarte 1972] and split impulse / NGS position passes [Catto 2014, 2024] hide bounce-from-correction but do not tax PE work of the remaining Δh ; TGS, soft constraints, and XPBD [Catto 2011; NVIDIA Corporation 2024; Macklin et al. 2016; Müller et al. 2007] damp or sidestep velocity-level bias at different cost. Variational integrators [Hairer et al. 2006] bound unconstrained drift; KRB is not one of those. To our knowledge, no prior single-pass SI setup applies an analytic PE/KE balance to expected Δh for both contacts and distance joints.

Two analytic sources inject artificial mechanical energy during constraint resolu-

tion.

Force-integration drift. With symplectic Euler integration, velocity is updated before the constraint solve:

$$v_{\text{solver}} = v_t + a_{\text{ext}} \cdot \Delta t. \quad (1)$$

Restitution and joint bias see v_{solver} instead of the true relative velocity v_{impact} . For a perfect bounce ($e = 1$), the launch velocity gains $a_{\text{ext}} \cdot \Delta t$ extra speed per frame. For a joint at rest, the solver sees gravity-induced velocity and fights it every frame at the velocity level.

Potential-energy shift from position correction. Baumgarte stabilization [Baumgarte 1972] feeds constraint error back as a velocity bias $v_{\text{bias}} = \beta C / \Delta t$, displacing bodies by Δh to resolve penetration or joint drift. That displacement increases potential energy (mgh) without reducing kinetic energy. The added PE converts to KE on later frames, causing runaway energy growth.

KRB applies that PE/KE balance to expected Δh at constraint setup for contacts and distance joints. The audit is per-constraint; coupled graphs can still offset (Section 6).

2. Method

KRB performs two independent corrections during constraint `preSolve`.

2.1. Component A: Force velocity compensation

With symplectic Euler, integration updates velocity before the constraint solve (Equation 1), so restitution and joint bias see v_{solver} rather than the pre-force relative velocity. Store the per-body force increment $v_{\text{force}} = a_{\text{ext}} \cdot \Delta t$ during integration and subtract it from the relative velocity seen at setup. That recovers the true impact normal velocity before restitution or bias is applied:

$$v_{\text{impact}} = v_{\text{relative}} - (v_{\text{force},B} - v_{\text{force},A}). \quad (2)$$

Component A applies to all constraints whenever symplectic Euler integration is used, independent of the bias method. In the implementation listing (Listing 1), `relativeVn` is this corrected normal velocity after subtracting `forceVn`.

2.2. Component B: Kinematic energy balancing

Baumgarte position correction displaces bodies by an expected distance Δh along the constraint normal. External forces do work $W = m(\mathbf{a}_{\text{ext}} \cdot \mathbf{n})\Delta h$ over that displacement. To keep mechanical energy consistent, the launch or bias speed must reflect that work: using $\Delta(\frac{1}{2}mv^2) = W$ gives a surface speed $v_{\text{surf}}^2 = v_{\text{impact}}^2 + 2(\mathbf{a}_{\text{ext}} \cdot \mathbf{n})\Delta h$.

When available kinetic energy cannot pay the tax, clamp with $\max(0, \cdot)$ and apply restitution:

$$v_{\text{surf}} = \sqrt{\max(0, v_{\text{impact}}^2 + 2(\mathbf{a}_{\text{ext}} \cdot \mathbf{n})\Delta h)}, \quad (3)$$

$$v_{\text{final}} = e \cdot v_{\text{surf}} = e \sqrt{\max(0, v_{\text{impact}}^2 + 2(\mathbf{a}_{\text{ext}} \cdot \mathbf{n})\Delta h)}. \quad (4)$$

For ground collisions ($\mathbf{a}_{\text{ext}} \cdot \mathbf{n} < 0$), Component B taxes launch velocity to pay for increased PE; for ceiling collisions ($\mathbf{a}_{\text{ext}} \cdot \mathbf{n} > 0$), it credits velocity for work against external forces.

2.3. Effective displacement prediction

Δh is the expected normal displacement used in the Component B work term—not the full penetration depth, but the portion the position solver will actually correct this step after accounting for launch motion and per-iteration caps. When velocity and position phases are decoupled, the kinematic bounce velocity v_{launch} (Listing 1: $v_{\text{Bounce}} = -e \cdot \text{relativeVn}$) partially resolves overlap after penetration slop s before the position solver runs. The remaining overlap after that motion is the effective depth

$$d_{\text{eff}} = \max(0, (d - s) - (v_{\text{launch}} \cdot \Delta t)). \quad (5)$$

Many engines clamp per-iteration correction (e.g. Gearbox2D `MAX_POSITION_CORRECTION`). The audit must match the work actually performed:

$$\Delta h = \min(d_{\text{eff}}, \Delta h_{\text{max}}) \cdot \Gamma, \quad (6)$$

where $\Gamma = 1 - (1 - \beta)^n$ is the cumulative Baumgarte fraction applied over n position iterations with factor β .

3. Constraint application

3.1. Contacts and speculative gaps

For physical overlap ($d > 0$), both Component A and Component B are required.

Speculative contacts are created before overlap ($d < 0$, a gap) to prevent tunneling. Because no position-correction work occurs yet, $\Delta h = 0$ and Component B is omitted; Component A remains critical so gravity-induced velocity is not treated as impact speed.

Apply restitution bias only when restitution is enabled and either the corrected normal velocity exceeds a small threshold, or a speculative gap is closing faster than the gap width allows. Resting contacts must not receive an $e = 1$ launch bias.

3.2. Distance joints and the zero-velocity paradox

Distance joints target zero relative velocity along the rod. Omitting Component B entirely leaks energy when Baumgarte stretch correction does work against gravity. Component B must be applied, but capped so a resting joint is not paralyzed.

Let $v_{\text{bias}} = \beta C / \Delta t$ be the ideal correction velocity for constraint error C . The audited correction is

$$v_{\text{bias.actual}} = \sqrt{\max(0, v_{\text{bias}}^2 - 2(\mathbf{a}_{\text{ext}} \cdot \mathbf{n})(\beta C))}. \quad (7)$$

Equation (7) is the same PE/KE audit with expected stretch displacement βC in place of contact Δh . If kinetic energy cannot pay the PE tax, the joint retains a microscopic Baumgarte sag—bounded audit rather than infinite stiffness at rest. This paper’s setup recipe covers contacts and *distance* joints; other joint types are outside the listed implementation.

4. Implementation

KRB runs at contact and distance-joint `preSolve`, before the velocity iteration loop. Prerequisites: symplectic Euler with per-body `forceVelocity` ($a_{\text{ext}} \Delta t$), Baumgarte position correction with factor β , position iteration count n , and penetration `slop`.

Integrate store. Per dynamic body at `integrate` (before the constraint solve): `forceVelocity` = `a_ext * dt` (two floats), with a_{ext} from gravity and external force before linear damping; zero for sleeping or infinite-mass bodies (supplemental `cpp/src/world/world-simd.cpp`).

4.1. Contact setup

```
forceVn = dot(bodyB.forceVelocity - bodyA.forceVelocity, n)
relativeVn = vn - forceVn
vBounce = -e * relativeVn
shouldBounce = enableRestitution &&
    (relativeVn < -RESTITUTION_THRESHOLD ||
     (depth < 0 && relativeVn < depth / dt))
if (shouldBounce) {
    if (depth > 0) {
        depthAfterVelocity = max(0, (depth - slop) - vBounce * dt)
        cumulativeFactor = 1 - pow(1 - beta, n)
        expectedDisplacement =
            min(depthAfterVelocity, MAX_POSITION_CORRECTION) * cumulativeFactor
    } else { expectedDisplacement = 0 }
    workTerm = 2 * (forceVn / dt) * expectedDisplacement
    vFinal = e * sqrt(max(0, relativeVn * relativeVn + workTerm))
    bias = (depth < 0) ? -max(vFinal, depth / dt) : -vFinal
} else if (depth < 0) { bias = -depth / dt } else { bias = 0 }
```

Listing 1. Contact preSolve bias (Component A + B).**4.2. Distance-joint setup**

```

forceVn = dot(bodyB.forceVelocity - bodyA.forceVelocity, n)
v_bias_ideal = beta * C / dt
expectedDisplacement = v_bias_ideal * dt
workTerm = 2 * (forceVn / dt) * expectedDisplacement
v_bias_sq = v_bias_ideal * v_bias_ideal
v_bias_actual = sqrt(max(0, v_bias_sq - workTerm))
bias = (v_bias_ideal > 0 ? v_bias_actual : -v_bias_actual) - forceVn

```

Listing 2. Distance-joint preSolve bias (Component A + B).

During the velocity solve, $-\text{forceVn}$ is folded into bias so `relative_vn` is not double-corrected (see `supplemental distance-joint.cpp`).

5. Evaluation

Four datasets (Table 1): A–B 600 s; C KRB-on 8 h ($t = 28800$ s); D `native world.step()` wall-time only. C01–C07: $dt = 1/60$, $e = 1$, zero friction/damping, sleep off. Datasets A and C use `stockWorld()` plus `constants.h: velocity.iterations=50, position.iterationsn=10,  $\beta = 0.2$ , MAX_POSITION_CORRECTION=0.2, slop 0.016, restitution threshold 0.01`; Listing 1 Γ uses that n, β . Dataset B engines are unmodified whole-engine WASM traces (not those Gearbox knobs). Full KRB on/off only (no A-only/B-only traces); C01–C03 do not isolate which leak each component closed. $E = E_k + E_p$ (A rotational, B translational only). E/E_0 ; 1 Hz (A–B) or 10 s (C) samples alias the bounce; 60 s windowed means are the drift signal; C07 uses 600 s windows on the enclosure series.

Table 1. Evaluation protocol and Dataset C (KRB on, 8 h runs). C07 windowed E/E_0 is on 600 s windows of the 8 h trace, not a 600 s run. Dataset D is ms/step native timing, not a Gearbox-versus-Box2D CPU comparison.

Set	Scene	Horizon	Surface	Outcome
A	all	600 s	log-energy	Table 2
B	floor, cradle	600 s	WASM benchmarks	Table 3
C	enclosure	8 h	log-energy	in box; 600 s win. $1.009 \rightarrow 0.977$
C	cradle	8 h	log-energy	$[1.034, 1.093]$; mean 1.053
D	A + stack	timed steps	native g++	Table 4

5.1. Cost (Dataset D)

Per dynamic body: store `forceVelocity =  $a_{\text{ext}} \cdot \Delta t$` . Per contact preSolve: `forceVn` dot, optional Γ , `workTerm`, one `sqrt`, bias update. Per distance joint: same pattern with $-\text{forceVn}$ folded into bias. Zero extra PGS, velocity, or position iterations.

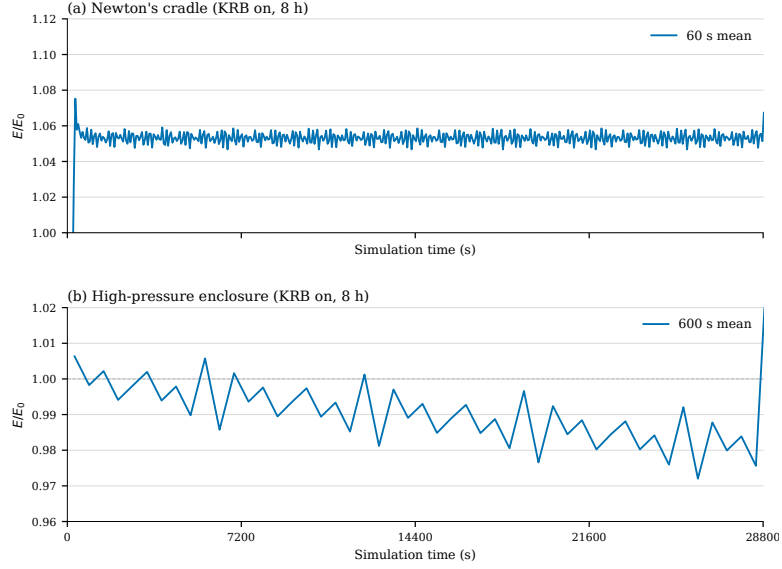


Figure 1. Dataset C KRB-on continuation to 8 h ($t = 28800$ s): windowed E/E_0 means (cradle 60 s, enclosure 600 s); not an on/off ablation. Empirical scene bounds, not global invariants; enclosure shows slow loss, not the KRB-off SI gain of Figure 2b.

Table 2. Dataset A ablation: E/E_0 at 600 s (compile-time KRB on/off).

Scene	KRB	E/E_0	Win. mean	Outcome
Floor bounce	on	0.999	1.000	bounded
Floor bounce	off	6.53	climbing	runaway
High-pressure circle	on	—	1.00	stayed in box
High-pressure circle	off	—	—	escaped, step 161
Newton's cradle	on	1.07	1.05 after $t = 300$	bounded offset
Newton's cradle	off	1.79	climbing	secular gain

6. Limitations

KRB is a per-constraint audit, not a coupled one; resolving one constraint can force work on another (e.g. horizontal collision lifting a pendulum rod).

Evaluation regime. In scope: single-pass SI/PGS, symplectic Euler (Component A not evaluated on other integrators), contacts and distance joints, $e = 1$, zero friction/damping, sleep off, and the `Gearbox World()/constants.h` knobs in Section 5. Dataset A/C cradle energy includes rotational E_k ; Dataset B cradle E/E_0 is translational only—not the same meter as Figure 2c (1.07) versus Table 3 Gearbox (1.062). Shown limits: coupled cradle $\sim 7\%$ step then $E/E_0 \in [1.034, 1.093]$ (mean 1.053) through 8 h (Figures 2c, 1a); C07 slow loss $1.009 \rightarrow 0.977$ (Figure 1b); speculative contacts omit Component B; joint sqrt cap leaves resting sag; KRB-off enclosure escape (Figure 2b) is SI leak, not KRB failure; unmodified Box2D/p2/Matter (Figure 3b) dissipate rather than gain. Out of scope: frictional or $e < 1$ games, hinges (engine-only), XPBD/TGS, global conservation. Variational integrators

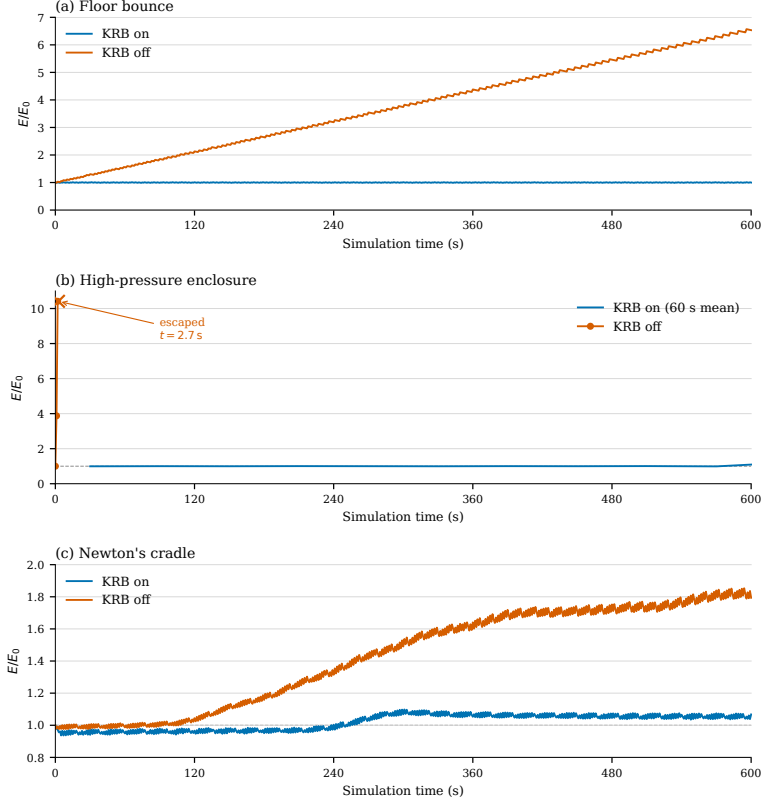


Figure 2. Mechanical energy ratio E/E_0 over 600 s. (a) Elastic floor contact. (b) High-rate enclosure; KRB on is a 60 s windowed mean, KRB off raw 1 Hz samples to tunneling ($t = 2.7$ s). (c) Newton's cradle.

Table 3. Dataset B: E/E_0 at 600 s on unmodified whole engines (cross-engine placement, not causal evidence that KRB explains Gearbox versus Box2D/p2/Matter; C01–C03 are the compile-time on/off ablation).

Scene	Engine	E/E_0	Outcome
Floor bounce	Gearbox2D (KRB on)	0.999	bounded
Floor bounce	Box2D-WASM	6.61	runaway
Floor bounce	p2.js	0.376	slow loss
Floor bounce	Matter.js	≈ 0	dead by $t \approx 160$ s
Newton's cradle	Gearbox2D (KRB on)	1.062	bounded offset
Newton's cradle	Box2D-WASM	0.221	stepped loss
Newton's cradle	p2.js	0.320	slow loss
Newton's cradle	Matter.js	≈ 0	dead by $t \approx 40$ s

[Hairer et al. 2006] are a different claim.

No A-only/B-only ablation; full KRB versus `-DGEARBOX_DISABLE_KRB` is the mea-

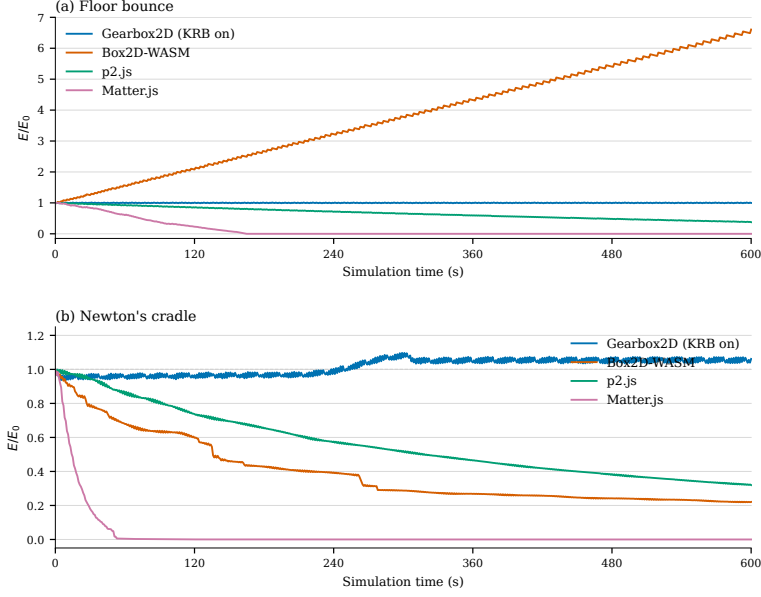


Figure 3. Mechanical energy ratio E/E_0 over 600 s on unmodified engines. (a) Elastic floor contact. (b) Newton's cradle. Gearbox2D is default KRB-on WASM; other engines are whole-engine traces, not in-engine KRB ablation.

Table 4. Dataset D: native KRB on/off wall-time per `world.step()` (mean and repeat range, ms/step). Whole-step timings may include hinge Component A (engine-only); Listings 1–2 and the analytic op-count above cover contacts and distance joints only (wall-time is scene-dependent; high-pressure enclosure shows a larger on/off spread than floor/cradle). BENCH_FLAGS: `-O3 -march=native -mfma -pthread -DGEARBOX_MT; 100 warmup + 1000 timed steps, three repeats (i9-9900KF, g++ 13.3.0, velocity_iterations= 50, position_iterations= 10)`. Compile-time ablation; not Box2D. Traces: `data/timing/timing-summary-{krb,nokrb}.csv`.

Scene	KRB on	KRB off	Notes
Floor bounce	0.010 (0.005–0.018)	0.009 (0.005–0.017)	Dataset A
High-pressure circle	0.012 (0.012–0.013)	0.006 (0.006–0.007)	Dataset A
Newton's cradle	0.048 (0.047–0.048)	0.049 (0.047–0.050)	Dataset A
Large stack	0.377 (0.372–0.381)	0.375 (0.369–0.380)	not Dataset A

sured switch. Dataset B is placement only (Section 5). Coupled-constraint audit, tunneling, and ill-conditioned chains remain future work. This note exceeds typical JCGT length (~ 4 pages) but stays within the 12-page ceiling.

7. Conclusion

KRB is a drop-in preSolve audit for velocity-level SI solvers [Catto 2005]: Component A compensates symplectic-Euler force drift; Component B taxes PE work of expected Δh , at a few scalars per constraint with zero extra PGS iterations (Table 4). The coupled-constraint gap in Section 6 remains open.

References

- BAUMGARTE, J. Stabilization of constraints and integrals of motion in dynamical systems. *Computer Methods in Applied Mechanics and Engineering*, 1(1):1–16, 1972. URL: [https://doi.org/10.1016/0045-7825\(72\)90018-7](https://doi.org/10.1016/0045-7825(72)90018-7). 1, 2
- BENDER, J., ERLEBEN, K., AND TRINKLE, J. Interactive simulation of rigid body dynamics in computer graphics. *Computer Graphics Forum*, 33(1):246–270, 2014. URL: <https://doi.org/10.1111/cgf.12272>. 1
- CATTO, E. Iterative dynamics with temporal coherence. Presented at the Game Developers Conference, San Francisco, CA, 2005. URL: https://box2d.org/files/ErinCatto_IterativeDynamics_GDC2005.pdf. 1, 9
- CATTO, E. Soft constraints. Presented at the Game Developers Conference, San Francisco, CA, 2011. URL: https://box2d.org/files/ErinCatto_SoftConstraints_GDC2011.pdf. 1
- CATTO, E. Understanding constraints. Presented at the Game Developers Conference, San Francisco, CA, 2014. URL: https://box2d.org/files/ErinCatto_UnderstandingConstraints_GDC2014.pdf. 1
- CATTO, E. Solver2d. Box2D blog post, February 2024. URL: <https://box2d.org/posts/2024/02/solver2d/>. 1
- HAIRER, E., LUBICH, C., AND WANNER, G. *Geometric Numerical Integration: Structure-Preserving Algorithms for Ordinary Differential Equations*, volume 31 of *Springer Series in Computational Mathematics*. Springer-Verlag, Berlin, Germany, 2 edition, 2006. URL: <https://doi.org/10.1007/3-540-30666-8>. 1, 7
- MACKLIN, M., MÜLLER, M., AND CHENTANEZ, N. XPBD: Position-based simulation of compliant constrained dynamics. In *Proc. 9th ACM SIGGRAPH Conference on Motion in Games (MIG)*, pages 49–54, Burlingame, CA, 2016. URL: <https://doi.org/10.1145/2994258.2994272>. 1
- MÜLLER, M., HEIDELBERGER, B., HENNIX, M., AND RATCLIFF, J. Position based dynamics. *Journal of Visual Communication and Image Representation*, 18(2):109–118, 2007. URL: <https://doi.org/10.1016/j.jvcir.2007.01.005>. 1
- NVIDIA CORPORATION. Rigid body dynamics. PhysX SDK 5.4 Documentation, 2024. URL: <https://nvidia-omniverse.github.io/PhysX/physx/5.4.1/docs/RigidBodyDynamics.html>. 1

Index of Supplemental Materials

Paths are relative to `studies/kinematic_restitution_balancing/` unless noted as repository-root (`cpp/...`).

- `jcgt/krb.tex`, `jcgt/krb.bib`, `jcgt/build-notes.md`.
- `KRB_Whitepaper.md` — number mirror (not authoritative).
- `figures/fig-energy-*.pdf`, `plot-energy-*.py` (including `plot-energy-dataset-c.py`).
- `data/README.md`, `data/*-krb`, `nokrb.csv`, `data/dataset-b-*600s.csv`, `data/dataset-c/`, `data/timing/`.
- `log-energy.cpp`, `log-timing.cpp`.
- `cpp/src/solvers/distance-joint.cpp`, `cpp/src/world/contact-constraint.cpp`.

Author Contact and License

Joshua Sideris, Gear3Games — josh.sideris@gmail.com, gearbox2d.com. Supplemental code and data are ISC-licensed; see LICENSE in the Gearbox2D repository (github.com/JSideris/Gearbox2D).

Joshua Sideris, Kinematic Restitution Balancing: A Per-Constraint Energy Audit for Velocity-Level Impulse Solvers, *Journal of Computer Graphics Techniques (JCGT)*, vol. ?, no. ?, 1–1, ???
 ???

Received: August 23, 2026

Recommended: ???-??-??

Published: ???-??-??

Corresponding Editor: ??? ???

Editor-in-Chief: Alexander Wilkie

© ??? Joshua Sideris (the Authors).

The Authors provide this document (the Work) under the Creative Commons CC BY-ND 4.0 license available online at <http://creativecommons.org/licenses/by-nd/4.0/>. The Authors further grant permission for reuse of images and text from the first page of the Work, provided that the reuse is for the purpose of promoting and/or summarizing the Work in scholarly venues and that any reuse is accompanied by a scientific citation to the Work.

